

Aporte Parte Práctica

Sistemas Distribuidos

Jhinson Stalyn Aucatoma Celorio
7mo Software “A”

Martes 2 de Junio del 2026

Link al repositorio: https://github.com/JhinsonAucatoma/Sistemas_distribuidos/tree/main/Prueba/demo1

Índice

1. Estructura del Proyecto	2
2. Parte A — Comunicación entre Procesos	2
2.1. ServidorTCP.java	2
2.2. ClienteTCP.java / Nodo2 / Nodo3	5
3. Parte B — Relojes Lógicos de Lamport	9
3.1. LamportClock.java	9
4. Parte D — Coordinación	11
4.1. BullyElection.java	11

1. Estructura del Proyecto

El proyecto se organiza dentro del paquete `ec.edu.uteq.distribuidas.socket` y contiene seis clases. La siguiente tabla resume qué hace cada una:

Cuadro 1: Clases del proyecto y su función principal

Clase	Función principal
<code>ServidorTCP.java</code>	Nodo principal. Escucha conexiones entrantes de clientes y nodos réplica, y gestiona cada una en un hilo propio.
<code>ClienteTCP.java</code>	Cliente interactivo. Se conecta al servidor, permite escribir comandos y muestra las respuestas en tiempo real.
<code>Nodo2.java</code>	Segunda réplica del cliente. Simula un segundo nodo que participa en la red distribuida.
<code>Nodo3.java</code>	Tercera réplica del cliente. Comportamiento idéntico a <code>Nodo2</code> .
<code>LamportClock.java</code>	Implementa el reloj lógico de Lamport con las tres reglas: evento interno, envío y recepción de mensajes.
<code>BullyElection.java</code>	Implementa el algoritmo Bully para la elección de coordinador entre procesos distribuidos.

2. Parte A — Comunicación entre Procesos

La comunicación entre cliente y servidor se realizó mediante sockets TCP. El servidor queda a la espera de conexiones en el puerto 9000, y cualquier cliente que se conecte puede intercambiar mensajes con él de forma inmediata.

Para evitar que dos mensajes enviados seguidos se mezclen, cada mensaje se delimita con un salto de línea (`\n`). El servidor lee línea a línea con `readLine()`, así siempre procesa un mensaje completo antes de pasar al siguiente.

2.1. ServidorTCP.java

Configuración inicial

Se definen tres constantes al arrancar: el número de puerto (9000), el máximo de clientes simultáneos (50) y un contador atómico que asigna un ID único a cada cliente que se conecta. El contador atómico es importante porque varios hilos lo incrementan al mismo tiempo sin colisionar entre ellos.

Código Java

```
1 private static final int PUERTO = 9000;
2 private static final int MAX_CLIENTES = 50;
3 private static final AtomicInteger contadorClientes = new AtomicInteger
  (0);
```

Bucle principal de aceptación

Dentro del método `main`, el servidor crea un pool de hasta 50 hilos y entra en un bucle infinito esperando conexiones. Cada vez que llega un nuevo cliente, `servidor.accept()` desbloquea, se le asigna un número de ID y se lanza un hilo dedicado (`ManejadorCliente`) para atenderle. El servidor principal vuelve al bucle inmediatamente, listo para el siguiente cliente.

```
1 while (!Thread.currentThread().isInterrupted()) {
2     Socket clienteSocket = servidor.accept();
3     int id = contadorClientes.incrementAndGet();
4     pool.submit(new ManejadorCliente(clienteSocket, id));
5 }
```

Procesamiento de mensajes por cliente

Cada hilo `ManejadorCliente` lee mensajes del cliente uno a uno con `readLine()`. Según lo que llegue, el servidor responde diferente: si recibe «HORA» devuelve la hora actual; si recibe «SALIR» se despide y cierra la conexión; cualquier otro texto lo devuelve como eco etiquetado con el ID del cliente. Si el cliente se desconecta de golpe, la excepción `IOException` se captura y el hilo se cierra limpiamente sin afectar a los demás clientes.

```
1 while ((linea = entrada.readLine()) != null) {
2     String respuesta = procesarMensaje(linea);
3     salida.println(respuesta);
4     if ("SALIR".equalsIgnoreCase(linea.trim())) break;
5 }
6
7 // Lógica de respuesta:
8 if ("HORA".equalsIgnoreCase(mensaje)) return "HORA_ACTUAL: " +
9     LocalDateTime.now();
10 if ("SALIR".equalsIgnoreCase(mensaje)) return "ADIOS: hasta luego
11     cliente #" + idCliente;
12 return "ECO [#" + idCliente + "]: " + mensaje;
```

Código Completo:

```
1 package ec.edu.uteq.distribuidas.socket;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.io.PrintWriter;
7 import java.net.ServerSocket;
8 import java.net.Socket;
9 import java.time.LocalDateTime;
10 import java.time.format.DateTimeFormatter;
11 import java.util.concurrent.ExecutorService;
12 import java.util.concurrent.Executors;
13 import java.util.concurrent.atomic.AtomicInteger;
14 import java.util.logging.Logger;
15
16 public class ServidorTCP {
17
18     private static final int PUERTO = 9000;
```

```

19 private static final int MAX_CLIENTES = 50;
20 private static final Logger logger = Logger.getLogger(ServidorTCP.
    class.getName());
21 private static final AtomicInteger contadorClientes = new
    AtomicInteger(0);
22
23 public static void main(String[] args) {
24
25     ExecutorService pool = Executors.newFixedThreadPool(MAX_CLIENTES
        );
26     logger.info("Iniciando servidor en puerto " + PUERTO);
27
28     try (ServerSocket servidor = new ServerSocket(PUERTO)) {
29         servidor.setReuseAddress(true);
30         System.out.println("[ " + timestamp() + " ] Servidor listo en
            : " + PUERTO);
31
32         while (!Thread.currentThread().isInterrupted()) {
33             Socket clienteSocket = servidor.accept();
34             int id = contadorClientes.incrementAndGet();
35             System.out.printf("[%s] Cliente #%d conectado desde %s %
                n",
36                 timestamp(), id, clienteSocket.
                    getRemoteSocketAddress());
37             pool.submit(new ManejadorCliente(clienteSocket, id));
38         }
39
40     } catch (IOException e) {
41         logger.severe("Error en servidor : " + e.getMessage());
42     } finally {
43         pool.shutdown();
44     }
45 }
46
47 private static String timestamp() {
48     return LocalDateTime.now()
49         .format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
50 }
51
52 static class ManejadorCliente implements Runnable {
53
54     private final Socket socket;
55     private final int idCliente;
56
57     ManejadorCliente(Socket socket, int idCliente) {
58         this.socket = socket;
59         this.idCliente = idCliente;
60     }
61
62     @Override
63     public void run() {
64         String nombreHilo = "Cliente-" + idCliente;
65         Thread.currentThread().setName(nombreHilo);
66
67         try (
68             BufferedReader entrada = new BufferedReader(
69                 new InputStreamReader(socket.getInputStream()));
70             PrintWriter salida = new PrintWriter(

```

```

71         socket.getOutputStream(), true)
72     ) {
73         String linea;
74         while ((linea = entrada.readLine()) != null) {
75             System.out.printf("[%s][%s] Recibido : %s %n",
76                 ServidorTCP.timestamp(), nombreHilo, linea);
77             String respuesta = procesarMensaje(linea);
78             salida.println(respuesta);
79             if ("SALIR".equalsIgnoreCase(linea.trim())) {
80                 break;
81             }
82         }
83     } catch (IOException e) {
84         System.err.println "[" + nombreHilo + "] Desconectado : "
85             + e.getMessage());
86     } finally {
87         cerrarSocket();
88     }
89
90     private String procesarMensaje(String mensaje) {
91         if ("HORA".equalsIgnoreCase(mensaje.trim())) {
92             return "HORA_ACTUAL : " + LocalDateTime.now();
93         }
94         if ("SALIR".equalsIgnoreCase(mensaje.trim())) {
95             return "ADIOS : hasta luego cliente #" + idCliente;
96         }
97         return "ECO [# " + idCliente + "]: " + mensaje;
98     }
99
100     private void cerrarSocket() {
101         try {
102             socket.close();
103         } catch (IOException ignored) {
104         }
105     }
106 }
107 }

```

2.2. ClienteTCP.java / Nodo2 / Nodo3

Los tres archivos son funcionalmente iguales: cada uno representa un proceso (cliente o nodo réplica) que se conecta al **ServidorTCP**. Al ejecutarse, el programa abre un socket hacia `localhost:9000`, muestra un prompt «>» al usuario y espera que escriba un comando. Ese texto se envía al servidor, se espera la respuesta y se muestra en pantalla. El ciclo se repite hasta que el usuario escribe **SALIR**.

Conexión y canal de comunicación

Se crean tres recursos de forma automática y segura usando `try-with-resources`: el socket TCP, un lector de líneas (`BufferedReader`) para recibir respuestas del servidor y un escritor (`PrintWriter` con auto-flush activado) para enviar mensajes. Al salir del bloque, Java cierra los tres recursos aunque ocurra un error.

```

1 Socket socket = new Socket(HOST, PUERTO);

```

```

2  BufferedReader entrada = new BufferedReader(new InputStreamReader(socket
    .getInputStream()));
3  PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);

```

Bucle de interacción con el usuario

El cliente entra en un bucle donde lee lo que escribe el usuario por teclado (Scanner), lo envía al servidor con `salida.println()` y espera la respuesta con `entrada.readLine()`. Si la respuesta empieza con «ADIOS», el bucle termina y la conexión se cierra. Si el servidor no está disponible al conectar, se muestra un mensaje de error claro indicando que el servidor no está corriendo.

```

1  while (true) {
2      String mensaje = teclado.nextLine();
3      salida.println(mensaje);           // enviar
4      String respuesta = entrada.readLine(); // esperar respuesta
5      System.out.println("Servidor: " + respuesta);
6      if (respuesta != null && respuesta.startsWith("ADIOS")) break;
7  }

```

Código Completo

```

1  package ec.edu.uteq.distribuidas.socket;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.io.PrintWriter;
7  import java.net.ConnectException;
8  import java.net.Socket;
9  import java.util.Scanner;
10
11  public class ClienteTCP {
12
13      private static final String HOST = "localhost";
14      private static final int PUERTO = 9000;
15
16      public static void main(String[] args) {
17
18          System.out.println("=== Cliente TCP -- Aplicaciones Distribuidas
              ===");
19
20          try (
21              Socket socket = new Socket(HOST, PUERTO);
22              BufferedReader entrada = new BufferedReader(
23                  new InputStreamReader(socket.getInputStream()));
24              PrintWriter salida = new PrintWriter(
25                  socket.getOutputStream(), true);
26              Scanner teclado = new Scanner(System.in)
27          ) {
28              System.out.println("Conectado a " + HOST + ":" + PUERTO);
29              System.out.println("Comandos : HORA | SALIR | cualquier
                  texto");
30              System.out.println("-----
                  ");
31

```

```

32         while (true) {
33             System.out.print("> ");
34             String mensaje = teclado.nextLine();
35             salida.println(mensaje); // enviar al servidor
36             String respuesta = entrada.readLine(); // esperar
               respuesta
37             System.out.println("Servidor : " + respuesta);
38             if (respuesta != null && respuesta.startsWith("ADIOS"))
               {
39                 System.out.println("Desconectando...");
40                 break;
41             }
42         }
43
44     } catch (ConnectException e) {
45         System.err.println(
46             "ERROR: No se pudo conectar a "
47             + HOST
48             + ":"
49             + PUERTO
50             + ". Est á el servidor corriendo?"
51         );
52     } catch (IOException e) {
53         System.err.println(
54             "Error de comunicación: "
55             + e.getMessage()
56         );
57     }
58 }
59 }

```

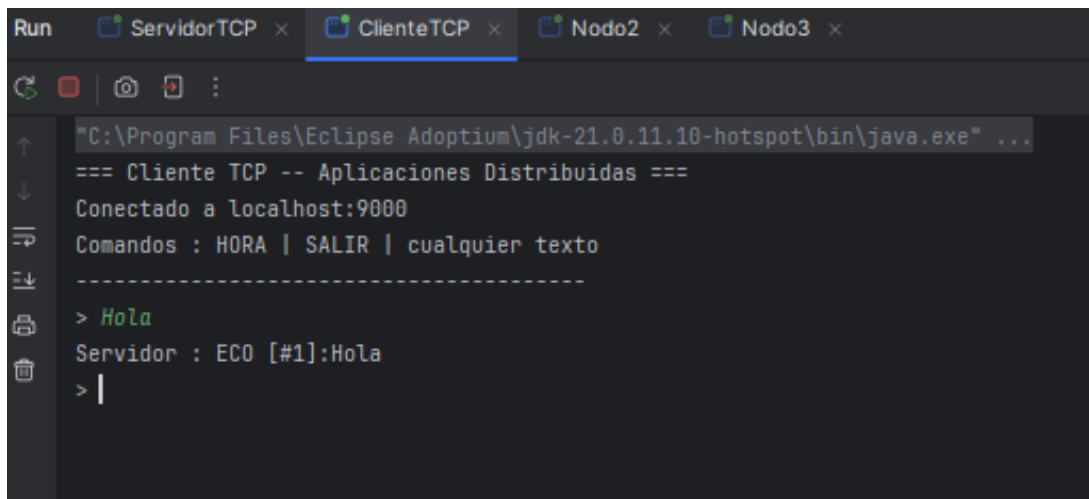
Capturas

```

Run  ServidorTCP x ClienteTCP x Nodo2 x Nodo3 x
C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe" ...
jun 02, 2026 3:44:09 P. M. ec.edu.uteq.distribuidas.socket.ServidorTCP main
INFORMACIÓN: Iniciando servidor en puerto 9000
[15:44:09.580] Servidor listo en :9000
[15:44:15.212] Cliente #1 conectado desde /127.0.0.1:63296
[15:44:17.206] Cliente #2 conectado desde /127.0.0.1:63300
[15:44:19.075] Cliente #3 conectado desde /127.0.0.1:63304

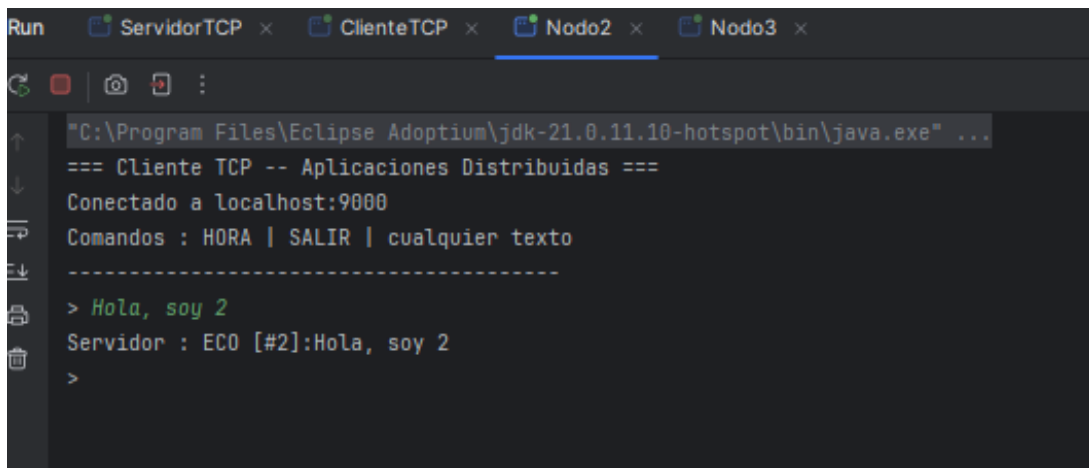
```

Figura 1: Servidor TCP — ejecución



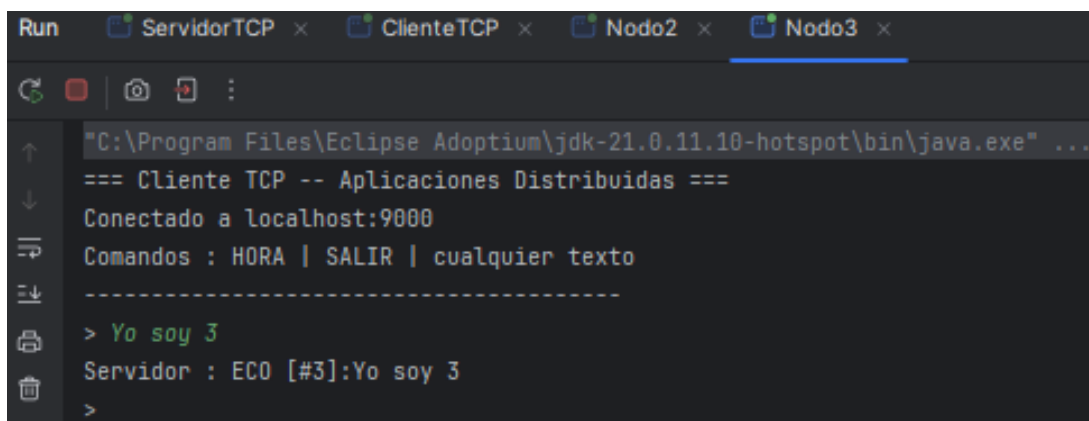
```
Run  ServidorTCP x  ClienteTCP x  Nodo2 x  Nodo3 x
C:\Program Files\Eclipse Adoptium\jdk-21.0.11.10-hotspot\bin\java.exe" ...
=== Cliente TCP -- Aplicaciones Distribuidas ===
Conectado a localhost:9000
Comandos : HORA | SALIR | cualquier texto
-----
> Hola
Servidor : ECO [#1]:Hola
> |
```

Figura 2: Cliente TCP — ejecución



```
Run  ServidorTCP x  ClienteTCP x  Nodo2 x  Nodo3 x
C:\Program Files\Eclipse Adoptium\jdk-21.0.11.10-hotspot\bin\java.exe" ...
=== Cliente TCP -- Aplicaciones Distribuidas ===
Conectado a localhost:9000
Comandos : HORA | SALIR | cualquier texto
-----
> Hola, soy 2
Servidor : ECO [#2]:Hola, soy 2
>
```

Figura 3: Nodo2 — ejecución



```
Run  ServidorTCP x  ClienteTCP x  Nodo2 x  Nodo3 x
C:\Program Files\Eclipse Adoptium\jdk-21.0.11.10-hotspot\bin\java.exe" ...
=== Cliente TCP -- Aplicaciones Distribuidas ===
Conectado a localhost:9000
Comandos : HORA | SALIR | cualquier texto
-----
> Yo soy 3
Servidor : ECO [#3]:Yo soy 3
>
```

Figura 4: Nodo3 — ejecución

Figura 5: Conexión entre procesos

3. Parte B — Relojes Lógicos de Lamport

3.1. LamportClock.java

La clase encapsula un contador `AtomicInteger` e implementa las tres reglas del algoritmo. Cada llamada imprime en consola el estado del reloj para que quede un registro visible del orden de eventos.

Regla 1 — Evento interno

Antes de que un proceso realice cualquier acción propia (sin enviar ni recibir nada), incrementa su contador en 1. Esto refleja que el tiempo avanza localmente.

```
1 public int eventoInterno(String descripcion) {
2     int t = contador.incrementAndGet(); // +1 antes del evento
3     System.out.printf("[%s] EVENTO INTERNO %-15s C=%d\n",
4                       nombreProceso, descripcion, t);
5     return t;
6 }
```

Regla 2 — Envío de mensaje

Justo antes de enviar un mensaje a otro proceso, el contador se incrementa en 1 y ese valor (la marca de tiempo) viaja dentro del mensaje. El receptor sabrá cuándo fue enviado según el reloj del emisor.

```
1 public int enviar(String destino, String mensaje) {
2     int t = contador.incrementAndGet(); // +1 antes de enviar
3     // t viaja dentro del mensaje como marca de tiempo
4     System.out.printf("[%s] ENVIO -> %s msg='%s' C=%d\n",
5                       nombreProceso, destino, mensaje, t);
6     return t;
7 }
```

Regla 3 — Recepción de mensaje

Al recibir un mensaje con una marca de tiempo, el proceso actualiza su reloj al mayor entre su valor actual y la marca recibida, y luego le suma 1. Así se asegura que el evento de recepción siempre tenga un valor mayor al del envío, respetando la causalidad. Se usa un bucle `compareAndSet` para que la actualización sea atómica y segura con múltiples hilos.

```
1 public int recibir(String origen, int marcaMensaje, String mensaje) {
2     int nuevo;
3     while (true) {
4         int actual = contador.get();
5         nuevo = Math.max(actual, marcaMensaje) + 1; // regla principal
6         if (contador.compareAndSet(actual, nuevo)) break;
7     }
8     System.out.printf("[%s] RECEPCION <- %s marcaMsg=%d C=%d%n",
9                       nombreProceso, origen, marcaMensaje, nuevo);
10    return nuevo;
11 }
```

Código Completo

```
1 package ec.edu.uteq.distribuidas.socket;
2
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class LamportClock {
6
7     private final AtomicInteger contador = new AtomicInteger(0);
8     private final String nombreProceso;
9
10    public LamportClock(String nombre) {
11        this.nombreProceso = nombre;
12    }
13
14    /** Regla 1: incrementar antes de evento interno. */
15    public int eventoInterno(String descripcion) {
16        int t = contador.incrementAndGet();
17        System.out.printf("[%s] EVENTO INTERNO %-15s C=%d%n",
18                          nombreProceso, descripcion, t);
19        return t;
20    }
21
22    /** Regla 2: incrementar y adjuntar marca al enviar. */
23    public int enviar(String destino, String mensaje) {
24        int t = contador.incrementAndGet();
25        System.out.printf("[%s] ENVIO -> %-8s msg='%-20s' C=%d%n",
26                          nombreProceso, destino, mensaje, t);
27        return t; // esta marca viaja con el mensaje
28    }
29
30    /** Regla 3: max(local, recibido) + 1 al recibir. */
31    public int recibir(String origen, int marcaMensaje, String mensaje)
32    {
33        int nuevo;
34        while (true) {
35            int actual = contador.get();
```

```

35         nuevo = Math.max(actual, marcaMensaje) + 1;
36         if (contador.compareAndSet(actual, nuevo)) break;
37     }
38     System.out.printf("[%s] RECEPCION <- %-6s msg='%-20s' marcaMsg=%
39         d C=%d%n",
40         nombreProceso, origen, mensaje, marcaMensaje, nuevo);
41     return nuevo;
42 }
43 public int getContador() { return contador.get(); }
44 public String getNombre() { return nombreProceso; }
45 }

```

4. Parte D — Coordinación

4.1. BullyElection.java

Estructura de un Proceso

Cada proceso conoce su propio ID, el total de procesos, y los buzones de todos. Tiene tres banderas: **activo** (si está corriendo), **liderActual** (quién es el coordinador ahora) y **enEleccion** (si hay una votación en curso). El campo estático **liderId** es compartido por todos y almacena el ganador final.

```

1 static class Proceso implements Runnable {
2     final int id;
3     final int totalProcesos;
4     final BlockingQueue<Mensaje>[] buzones;
5     volatile boolean activo;
6     volatile int liderActual = -1;
7     volatile boolean enEleccion = false;
8     static final AtomicInteger liderId = new AtomicInteger(-1);
9 }

```

Tipos de mensaje

El algoritmo usa solo tres tipos de mensaje. **ELECTION** lo lanza un proceso que quiere ser coordinador y lo manda a todos los de ID mayor. **OK** lo responde cualquier proceso de mayor ID, significando ‘yo también estoy vivo, no te proclames líder todavía’. **COORDINATOR** lo envía el ganador a todos para anunciar que él es el nuevo jefe.

Cuadro 2: Tipos de mensaje del algoritmo Bully

Mensaje	Significado
ELECTION	Un proceso lanza su candidatura y desafía a los de mayor ID a que respondan.
OK	Respuesta de un proceso con ID mayor: ‘sigu vivo, no te proclames líder’.
COORDINATOR	El ganador anuncia a todos que él es el nuevo coordinador.

Inicio de una elección

Cuando un proceso quiere ser coordinador porque detectó que el líder actual cayó, llama a `iniciarEleccion()`. Este método envía un mensaje `ELECTION` a cada proceso con ID mayor que él. Si no hay nadie con ID mayor, se proclama líder directamente. Si envió mensajes y nadie responde OK en 1.5 segundos, también se proclama líder.

```
1 void iniciarEleccion() {
2     enEleccion = true;
3     boolean hayMasGrande = false;
4
5     for (int i = id + 1; i < totalProcesos; i++) {
6         enviar(TipoMensaje.ELECTION, i); // desafía a los de mayor ID
7         hayMasGrande = true;
8     }
9
10    if (!hayMasGrande) {
11        proclamarLider(); // soy el de mayor ID, gana directamente
12    } else {
13        // esperar 1.5s, si nadie responde OK -> me proclamo líder
14        CompletableFuture.delayedExecutor(1500, TimeUnit.MILLISECONDS)
15            .execute(() -> { if (enEleccion) proclamarLider(); });
16    }
17 }
```

Proclamar líder y notificar a todos

Cuando un proceso gana la elección, llama a `proclamarLider()`: actualiza su propia variable `liderActual`, registra el resultado en el campo estático `liderId` y envía un mensaje `COORDINATOR` a todos los demás procesos para que actualicen su registro del líder actual.

```
1 void proclamarLider() {
2     liderActual = id;
3     liderId.set(id);
4     enEleccion = false;
5     for (int i = 0; i < totalProcesos; i++) {
6         if (i != id) enviar(TipoMensaje.COORDINATOR, i);
7     }
8 }
```

Código Completo

```
1 package ec.edu.uteq.distribuidas.socket;
2
3 import java.util.concurrent.*;
4 import java.util.concurrent.atomic.AtomicInteger;
5
6 public class BullyElection {
7
8     enum TipoMensaje { ELECTION, OK, COORDINATOR }
9
10    record Mensaje(TipoMensaje tipo, int emisor, int destino) {}
11
12    static class Proceso implements Runnable {
13        final int id;
```

```

14     final int totalProcesos;
15     final BlockingQueue<Mensaje>[] buzones;
16     volatile boolean activo;
17     volatile int liderActual = -1;
18     volatile boolean enEleccion = false;
19     static final AtomicInteger liderId = new AtomicInteger(-1);
20
21     @SuppressWarnings("unchecked")
22     Proceso(int id, int total, BlockingQueue<Mensaje>[] buzones,
23         boolean activo) {
24         this.id = id;
25         this.totalProcesos = total;
26         this.buzones = buzones;
27         this.activo = activo;
28     }
29
30     @Override
31     public void run() {
32         Thread.currentThread().setName("Proceso-" + id);
33         System.out.printf("[P%d] Iniciado. Activo=%b\n", id, activo);
34
35         while (activo) {
36             try {
37                 Mensaje msg = buzones[id].poll(500, TimeUnit.
38                     MILLISECONDS);
39                 if (msg != null) procesarMensaje(msg);
40             } catch (InterruptedException e) {
41                 Thread.currentThread().interrupt();
42                 break;
43             }
44         }
45         System.out.printf("[P%d] Finalizado.\n", id);
46     }
47
48     void procesarMensaje(Mensaje msg) {
49         switch (msg.tipo()) {
50             case ELECTION -> {
51                 System.out.printf("[P%d] Recibe ELECTION de P%d\n",
52                     id, msg.emisor());
53                 // Responder OK al emisor (soy mas grande)
54                 enviar(TipoMensaje.OK, msg.emisor());
55                 // Iniciar mi propia eleccion si no la tengo activa
56                 if (!enEleccion) iniciarEleccion();
57             }
58             case OK -> {
59                 System.out.printf("[P%d] Recibe OK de P%d -> alguien
60                     mas grande existe\n",
61                     id, msg.emisor());
62                 enEleccion = false; // otro tomara el control
63             }
64             case COORDINATOR -> {
65                 liderActual = msg.emisor();
66                 liderId.set(liderActual);
67                 enEleccion = false;
68                 System.out.printf("[P%d] NUEVO LIDER: P%d\n", id,
69                     liderActual);
70             }
71         }
72     }

```

```

66     }
67 }
68
69 void iniciarEleccion() {
70     enEleccion = true;
71     System.out.printf("[P%d] Iniciando ELECTION...\n", id);
72     boolean hayMasGrande = false;
73
74     // Enviar ELECTION a todos los procesos con ID mayor
75     for (int i = id + 1; i < totalProcesos; i++) {
76         enviar(TipoMensaje.ELECTION, i);
77         hayMasGrande = true;
78     }
79
80     // Si no hay nadie mas grande, me proclamo lider
81     if (!hayMasGrande) {
82         proclamarLider();
83     } else {
84         // Esperar respuesta OK (timeout 1.5s)
85         CompletableFuture.delayedExecutor(1500, TimeUnit.
            MILLISECONDS)
86             .execute(() -> {
87                 if (enEleccion) {
88                     proclamarLider();
89                 }
90             });
91     }
92 }
93
94 void proclamarLider() {
95     liderActual = id;
96     liderId.set(id);
97     enEleccion = false;
98     System.out.printf("[P%d] Me proclamo LIDER! Enviando
99     COORDINATOR a todos.\n", id);
100     for (int i = 0; i < totalProcesos; i++) {
101         if (i != id) enviar(TipoMensaje.COORDINATOR, i);
102     }
103 }
104
105 void enviar(TipoMensaje tipo, int destino) {
106     if (destino < totalProcesos) {
107         try {
108             buzones[destino].put(new Mensaje(tipo, id, destino))
109             ;
110         } catch (InterruptedException e) {
111             Thread.currentThread().interrupt();
112         }
113     }
114 }
115
116 void simularFallo() {
117     System.out.printf("[P%d] *** SIMULANDO FALLO ***\n", id);
118     activo = false;
119 }
120
121 @SuppressWarnings("unchecked")
122 public static void main(String[] args) throws Exception {

```

```

121     final int N = 5;
122     BlockingQueue<Mensaje>[] buzones = new BlockingQueue[N];
123     for (int i = 0; i < N; i++) {
124         buzones[i] = new LinkedBlockingQueue<>();
125     }
126
127     // Crear procesos (todos activos inicialmente)
128     Proceso[] procs = new Proceso[N];
129     for (int i = 0; i < N; i++) {
130         procs[i] = new Proceso(i, N, buzones, true);
131     }
132
133     ExecutorService exec = Executors.newFixedThreadPool(N);
134     for (Proceso p : procs) exec.submit(p);
135
136     // Inicialmente P4 (id=4, el mayor) es lider
137     procs[4].proclamarLider();
138     Thread.sleep(1000);
139
140     System.out.println("\n--- Simulando fallo del lider P4 ---\n");
141     procs[4].simularFallo();
142
143     // P1 detecta el fallo e inicia eleccion
144     Thread.sleep(500);
145     procs[1].iniciarEleccion();
146
147     Thread.sleep(3000);
148     System.out.printf("%nLider final elegido: P%d%n",
149         Proceso.liderId.get());
150
151     exec.shutdownNow();
152 }
153 }
154 }

```

Revisión de intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	AUCATOMA CELORIO JHINSON STALYN
Comenzado el	Martes, 2 de Junio de 2026, 07:57
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:07
Tiempo empleado	0:10:00
Calificación obtenida	8.25 / 20.00
Calificación final	4.13 / 10.00

PREGUNTA

1

Finalizado

0.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que arriesgarse a que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

- Seleccione una alternativa:
- ☐ a. Consistencia y disponibilidad (CA)
 - ☐ b. Consistencia y tolerancia a particiones (CP)
 - ☐ c. Las tres propiedades simultáneamente
 - ☒ d. Disponibilidad y tolerancia a particiones (AP). ✖

La respuesta correcta es: Consistencia y tolerancia a particiones (CP)

PREGUNTA

2

Finalizado

0.00 / 1.00

Un equipo intenta mejorar el rendimiento de una aplicación que envía miles de mensajes muy pequeños entre dos nodos. Duplica el ancho de banda del enlace, pero el rendimiento prácticamente no cambia. ¿Qué falacia de la computación distribuida explica este resultado?

- Seleccione una alternativa:
- ☐ a. "El transporte es gratuito".
 - ☐ b. "La latencia es cero".
 - ☐ c. "La red es confiable"
 - ☒ d. "El ancho de banda es infinito". ✖

La respuesta correcta es: "La latencia es cero".

PREGUNTA

3

Finalizado

1.00 / 1.00

Un equipo necesita invocar métodos sobre objetos remotos entre dos máquinas virtuales de Java, pasando objetos como parámetros y manejando automáticamente la serialización. ¿Qué mecanismo es el más apropiado y por qué?

- Seleccione una alternativa:
- ☐ a. Llamada a procedimiento remoto clásica estilo C, porque soporta orientación a objetos de forma nativa.
 - ☐ b. Sockets TCP crudos, porque ofrecen el mayor control de bajo nivel.
 - ☐ c. TCP, porque garantiza la entrega ordenada y fiable de los objetos
 - ☒ d. Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización. ✔

La respuesta correcta es: Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.

PREGUNTA

4

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?.

NAVEGACIÓN

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

Correcto

☐ Parcial

Incorrecto

Sistema de Gestión Académica (/)

Finalizado

1.00 / 1.00

PREGUNTA

5

Finalizado

1.00 / 1.00

PREGUNTA

6

Finalizado

1.00 / 1.00

PREGUNTA

7

Finalizado

1.00 / 1.00

PREGUNTA

8

Finalizado

0.00 / 1.00

PREGUNTA

9

Finalizado

0.00 / 1.00

Seleccione una alternativa:

☒ a. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes. ✓

☐ b. El sistema distribuido no admite fallos parciales.

☐ c. El sistema paralelo no puede tener múltiples procesadores.

☐ d. El sistema distribuido siempre procesa más rápido.

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

☐ a. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer.

☐ b. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.

☐ c. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.

☒ d. Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización. ✓

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de "me gusta" en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

☒ a. Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después. ✓

☐ b. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada

☐ c. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.

☐ d. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después.

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

¿Cuál enunciado distingue correctamente la "tolerancia a fallos" de la "prevención de fallos"?

Seleccione una alternativa:

☒ a. La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran. ✓

☐ b. La tolerancia a fallos elimina por completo todos los fallos posibles, mientras que la prevención pone en alerta al administrador sobre posibles fallos.

☐ c. Son términos sinónimos.

☐ d. La prevención de fallos solo es aplicable al hardware, mientras que la tolerancia a fallos es tanto al software como al hardware.

La respuesta correcta es: La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.

Un carrito de compras prioriza la disponibilidad (AP) y emplea consistencia eventual. Un usuario agrega un ítem y, al recargar de inmediato, no lo ve; segundos después sí aparece. ¿Cómo se explica esto correctamente?

Seleccione una alternativa:

☐ a. El sistema cambió automáticamente a un modo CP e indica que el sistema no es tolerante a particiones

☐ b. Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.

☐ c. Es un fallo del sistema que viola el teorema CAP.

☒ d. Es el comportamiento esperado: alta consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata devuelve datos actualizados. ✗

La respuesta correcta es: Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

☐ a. Transparencia de migración.

☐ b. Transparencia de acceso.

☒ c. Transparencia de ubicación. ✗

12

Universidad Técnica Estatal De Quevedo

© 2026 - Todos los derechos reservados

2:43 PM

La respuesta correcta es: Transparencia de migración.

PREGUNTA

10

Finalizado

1.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- ☐ a. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción
- ☒ b. Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "si" pueden quedar bloqueados indefinidamente esperando la decisión final. ✓
- ☐ c. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "si" quedan libres del bloqueo indefinidamente esperando la decisión final.
- ☐ d. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "si" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA

11

Finalizado

0.50 / 1.00

Relacione el escenario y tipo de transparencia evidenciada:

Relacione la opción correcta:

Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco.

Transparencia De Migración ✕

Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad.

Transparencia De Migración ✕

Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note.

Transparencia De Fallos ✓

Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban.

Transparencia De Concurrencia ✓

La respuesta correcta es: Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco. → Transparencia de persistencia , Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban. → Transparencia de concurrencia , Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad. → Transparencia de ubicación , Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note. → Transparencia de fallos

PREGUNTA

12

Finalizado

0.00 / 1.00

En los relojes lógicos de Lamport se cumple que si el evento *a* ocurre antes que *b* ($a \rightarrow b$), entonces $C(a) < C(b)$. Sin embargo, observar que $C(x) < C(y)$ NO permite concluir que $x \rightarrow y$. ¿Por qué?

Seleccione una alternativa:

- ☐ a. Porque Lamport exige relojes físicos perfectamente sincronizados, es decir, que deben dar los mismos valores en los mismos tiempos.
- ☐ b. Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.
- ☒ c. Porque $C(x) < C(y)$ siempre implica necesariamente $x \rightarrow y$, para la sincronización de los procesos. ✕
- ☐ d. Porque los relojes de Lamport no son monótonos crecientes.

La respuesta correcta es: Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.

PREGUNTA

13

Finalizado

0.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes "HOLA" y "MUNDO", pero en el receptor a veces llegan juntos como "HOLAMUNDO" en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ☒ a. El sistema cambió automáticamente a UDP. ✕
- ☐ b. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente
- ☐ c. TCP perdió y luego retransmitió un mensaje.
- ☐ d. Es un error físico de la tarjeta de red

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente

PREGUNTA

14

Finalizado

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- ☐ a. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales.

PREGUNTA

15

Finalizado

0.75 / 1.00

- ☒ b. La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados. ✓

☐ c. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.

☐ d. La integridad de los datos en tránsito

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

Relaciones: Protocolo o algoritmo de coordinación y su propósito clave

Relacione la opción correcta:

Commit en dos fases (2PC)

Garantiza Que Una Transacción Se Confirme En Todos Los Nodos O En Ninguno ✓

Consenso Raft / Paxos

Elegir... ✕

Algoritmo Bully

Elige Como Coordinador Al Nodo Activo De Mayor Identificador Tras Detectar Fallo ✓

Relojes de Lamport

Ordena Causalmente Los Eventos Sin Dependere De Un Reloj Físico Global ✓

La respuesta correcta es: Relojes de Lamport → Ordena causalmente los eventos sin depender de un reloj físico global. , Consenso Raft / Paxos → Logra acuerdo sobre un único valor mediante quórum de mayoría, tolerando caídas de nodos. , Algoritmo Bully → Elige como coordinador al nodo activo de mayor identificador tras detectar una caída. , Commit en dos fases (2PC) → Garantiza que una transacción se confirme en todos los nodos o en ninguno, aunque puede quedar bloqueado si el coordinador cae.

PREGUNTA

16

Sin contestar

0.00 / 1.00

Un acuerdo de nivel de servicio (SLA) promete una disponibilidad anual del 99,9 %. ¿Aproximadamente cuánto tiempo de inactividad permite al año, y qué implicaría elevarlo al 99,99 %?

Seleccione una alternativa:

- ☐ a. ≈ 8,76 horas al año; al 99,99 % se reduciría a ≈ 52,6 minutos al año.

☐ b. ≈ 88 horas al año; el 99,99 % lo empeoraría.

☐ c. ≈ 3,65 días al año; el 99,99 % no cambiaría nada.

☐ d. ≈ 1 minuto al año; el 99,99 % lo duplicaría.

La respuesta correcta es: ≈ 8,76 horas al año; al 99,99 % se reduciría a ≈ 52,6 minutos al año.

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.

Elegir... ▼

Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.

Elegir... ▼

El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.

Elegir... ▼

Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.

Elegir... ▼

La respuesta correcta es:

PREGUNTA

18

Sin contestar

0.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- ☐ a. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

☐ b. Cliente-servidor de tres capas, porque separa la lógica de negocio.

☐ c. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones.

☐ d. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

19

Sin contestar

0.00 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Elegir...

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Elegir...

Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Elegir...

Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Elegir...

La respuesta correcta es:

PREGUNTA

20

Sin contestar

0.00 / 1.00

Sockets TCP

Relacione la opción correcta:

Invocación remota de métodos (RMI)

Elegir...

Sockets UDP

Elegir...

Middleware orientado a mensajes (colas)

Elegir...

La respuesta correcta es: